QuickSend: A Low Overhead Software-Transparent Graphical Data Compression Algorithm

Alan Wang
alanlw2@illinois.edu
UIUC
USA

Kaustubh Khulbe
kkhulbe2@illinois.edu
UIUC
USA

Trisha Murali
tmurali2@illinois.edu
UIUC
USA

Jay Shah
jpshah3@illinois.edu
UIUC
USA

Abstract

With the end of Dennard scaling and the slowing of Moore's law, architects must rely on alternative means to meet the performance needs of applications such as AI. One major bottleneck faced by applications is memory bandwidth, which has not advanced as rapidly as processor compute power. A hardware optimization used to increase the effective memory bandwidth of processors is software-transparent, lossless compression schemes. Several of these hardware compression schemes have been implemented in widely used devices such as phones or desktops. However, these algorithms are often black boxes and comprehensive comparisons between them are lacking. In our work, we systematically evaluate three industry algorithms using our own testing harness to provide a focused comparison. We then present QuickSend, our compression scheme that builds on insights from our analysis, designed to improve upon one of the industry algorithms. Finally, we evaluate the area and power overheads of QuickSend against the industry algorithm, demonstrating our algorithm's performance improvements with negligible impacts on power.

1 Introduction

As we move further into an AI-dominated world, the demand for exponentially scaling compute capabilities continues to grow. To address this demand, architects have increasingly turned to optimizations spanning all sectors of hardware. In recent years, we have begun to see the emergence of more exotic prefetchers [2, 11, 13], silent stores [12], pipeline compression [12], computation reuse [12], value prediction [12], load address prediction [4], load output predictions [3], and more. However, these optimizations have ignored a critical bottleneck in modern processors: memory bandwidth. Specifically, CPU-to-GPU memory bandwidth.

Many latency-sensitive workloads, such as rendering and gaming, are dependent on the CPU-to-GPU memory bandwidth. Stalls due to limited bandwidth create noticeable lag that frustrates users and damages the overall user experience. Moreover, without continuous increases in GPU bandwidth, the performance gains attained through faster hardware diminish with each generation. To help alleviate this problem, researchers have turned to software-transparent lossless compression between the CPU and GPU. Since the compression is software-transparent, applications function without any awareness of it. Meanwhile, under the hood, data moving from the CPU to the GPU is compressed in hardware to reduce data movement costs. These compression algorithms are especially relevant for graphical workloads, where rendering quality directly affects user experience.

These compression algorithms are different from conventionally understood compression algorithms, where the goal is static space conservation. Moreover, classical compression algorithms aren't designed for random access and require full decompression to access the original data. In contrast, these CPU-to-GPU hardware compression algorithms often consume more space when at rest. Instead, these algorithms are built for random access and reduce the number of bits needed to move data around [14].

The significant impact of CPU-to-GPU bandwidth has led nearly all hardware vendors to adopt some variant of software-transparent GPU compression for CPU-to-GPU communication. Despite their widespread use, these industry algorithms have remained black boxes, with limited understanding of their inner workings until recent efforts to reveal them [14]. Comparisons among the industry algorithms, however, are still missing from the literature. Further, we believe additional optimizations can be added to the industry schemes to further improve performance. To address these gaps, in our work, we present the first evaluation of different industry compression schemes. We then present QuickSend, our own compression scheme that builds on a previous industry algorithm, leveraging insights from our analysis to further improve performance. Additionally, we evaluate the power and area overheads of our QuickSend architecture to show that these optimizations come with negligible overhead for a large gain in performance.

For our industry comparisons, we target two previously reverse-engineered Intel compression algorithms and a recently reverse-engineered, unpublished Arm Mali GPU compression algorithm. We develop a testing framework capable of evaluating these compression schemes across a diverse set of images, access patterns, and GPU cache sizes. This analysis then motivates our QuickSend design, which we similarly analyze in our testing harness.

In summary, our contributions are as follows:

- We implement three industry (two Intel and one Arm) compression algorithms in Python.
- We create a testing harness to assess the compressibility of the three industry compression algorithms as well as our own compression scheme.

- We analyze the compressibility of the three industry algorithms across a rigorous set of benchmarks and implement our own compression algorithm using insights from our analysis.
- We implement the RTL for the Intel 8th generation algorithm and our own QuickSend scheme to compare the area and power overheads of our compression algorithm to industry schemes.

2 Compression Algorithms

We now present an overview of the three industry algorithms evaluated along with a description of our own compression algorithm, motivated by the results shown in Section 5. These algorithms, along with an uncompressed baseline, serve as the basis for the remainder of this paper.

2.1 Intel 8th Gen.

The core principle behind the Intel iGPU 8th Gen. algorithm is to identify and store the minimum value in each channel and compute the residuals relative to that value [14]. If it is possible to fit all the *residuals* of data into a single cache line, compression is successful at a $2:1$ ratio.

The improvement in Intel iGPU 8th Gen. comes from representing pixels as offsets from the minimum value. The more homogeneous the block is, the more likely it is that compression is successful.

Let B be a three dimensional tensor with dimensions (C, W, H). The first step in the iGPU 8th Gen. algorithm is to compute the minimum value for each channel. That is, it generates a tensor of shape $M = (C, 1)$, where each entry at c is $\min(B[c, :, :])$. In effect, it min-reduces B across the C dimension.

From there, we compute the residuals. That is, the difference from each pixel to the minimum values. Every single value in this new tensor is ≥ 0. Let $R = B - \texttt{broadcast}(M, -1, -1)$.

The next step is to then compute the maximum residual we observe in R. That is, $M' = (C, 1)$, where each entry at c is now $\max(B[c, :, :])$.

Block B can be compressed if and only if

$$\sum_{i=0}^{C} \lceil \log_2(M'[i]) \rceil \leq 14$$

This is because each block has 32 pixels. To ensure that all pixels fit within a single 64-byte cache line, we need:

$$48 + BITS \cdot 32 \leq 64 \cdot 8 \rightarrow BITS \leq 14$$

Note that the 48 comes from the header. The header contains skip bits (one for each channel, set if the maximum residual is 0), the minimum RGBA values (i.e. M), and the bits required for each channel.

If B can be compressed, Intel iGPU 8th Gen. compresses 128 bytes of data into 64 bytes, or a single cache line. The algorithm chunks up the input image into cache lines, and each cache line is marked with a 2 bit flag denoting if it is uncompressed, if it is compressed and should be used, or if it is compressed and it's neighbor is the data that should be sent. Therefore, the algorithm *requires more data to store*, but *sends less data.*

2.2 Intel 11th Gen.

The compression scheme employed in the 11th generation SoCs builds upon the well-established prediction-plus-residual framework used in prior generations, while introducing several minor adjustments. The fixed header format has been revised to include a 1-bit color transform flag, followed by 8 skip bits typically set to zero — used only in special compression modes, 4 bits for each channel and a 32-bit prediction value — for a total header size of 53 bits.

Unlike previous generations that used the row-major pixel order (as in the legacy tile-Y layout), the 11th-generation format adopts 2×2 pixel blocks traversed in Z order, enhancing spatial locality for compression. Although uncompressed cachelines still maintain row-major order, the compressed layout leverages Z-order traversal for improved use of spacial locality. The compression setting most analogous to the 8th-generation format compresses 32 pixels from two cachelines into a single cacheline, allocating 14 bits per pixel. In contrast to earlier implementations that used 2-bit metadata per cacheline pair, the 11th-generation design utilizes a 4-bit metadata field, enabling support for multiple compression configurations. For this study we have used the 128 to 64 bytes compression mode.

The notable features distinguish the 11th-generation compression engine are as follows: Each compressed region consists of eight 2×2 pixel sub-blocks. If sub-blocks are uniform (i.e., all four pixels in a block share the same color), the number of required residuals is reduced from 32 to 20, thereby increasing the available bit budget to 22 bits per residual. The uniformity of blocks is encoded in bits 2 through 9 of the 53-bit header. These bits correspond to the eight 2×2 blocks in Z-order, where a bit value of 1 indicates a uniform block that can be represented by a single residual.

These modifications enable the 11th generation GPU compression system to dynamically adapt encoding precision, reduce storage requirements, and optimize memory bandwidth, all while maintaining lossless reconstruction fidelity under eligible conditions.

2.3 Arm Mali

The Arm Mali compression algorithm creates a data tree and a bit length tree for each channel to represent a 4x4 pixel pane. The data tree is a full three-layer tree, where each node in the tree has four children nodes. The bit length tree is a full two-layer tree, where again each node has four children nodes.

To construct the data tree, the value of each leaf node is set to a pixel value in the 4x4 pane. Each pixel is in a 2x2 quadrant (quad) of the pane, and all pixels in the quadrant share the same parent node in the 2nd level of the tree (the leaf nodes are in the 3rd level of the tree). The value of the 2nd level parent nodes is the minimum value of the 2nd level node's child nodes. The value of the 2nd level parent node is then subtracted from the value of all its children leaf nodes. Next, the root node (1st level node) is set to the minimum value among the four 2nd level nodes. The value of the root node is then subtracted from the 2nd level nodes. This process is then repeated for each of the channels to construct a data tree for each channel.

To construct the bit length tree, the max bit length of the quad is used for the root value of the bit length tree. The length of each leaf node is the max bit length of each set of leaf nodes in the data

tree (each quad has one set of leaf nodes). The root bit length tree value is then subtracted from each of the leaf nodes in the bit count tree. The root node bit length is 4 bits, and each leaf node is given 2 bits. One bit length tree is constructed per data tree and thus per channel of the 4x4 pane.

The exact criteria for when a 4x4 pane is compressed by the Mali compression algorithm is complex and left to the code submitted with our project. The general rule is if the compression pane is under 64 B, compression will occur. Otherwise, no compression occurs.

We, the authors, note that this Arm algorithm is taken from an upcoming publication where the Arm compression algorithm was reverse engineered. All credit for the Arm compression algorithm goes to the authors of said publication.

2.4 QuickSend

Using our evaluation (cf. Section 5) of the industry compression algorithms we create two improvements over the Intel 8th generation compression scheme. This is possible as the Intel 8th gen compression scheme leaves 16 additional bits in the header for further optimizations. We use these bits to provide the metadata needed for our optimizations.

The first modification we enable is one that allows the compression scheme to exploit cross channel redundancies in the compression block. For example, if the R and G channels of a compression block are identical, it makes no sense to transmit both set of pixel values. However, in the baseline Intel 8th generation, both channels would be transmitted, wasting bits. For our implementation, we only consider if there is redundancy between the R, G, and B channels. Only if all three channels transmit the same pixels do we enable our optimization. This only allows our optimization to exploit grayscale images by reducing the bits transmitting by 2x.

In the future, a better version of the above modification is to allow for other combinations of channels to be further optimized. For example, only considering the R and G channels or G and B channels. This opens more opportunities than the current version for removing inter-channel redundancies.

The second optimization we employ is an optimization that exploits colorspaces. Instead of just using the RGBA colorspace, this second optimization computes the compression block in both RGBA and YUVA and then selects the better compression scheme to use. This allows the compression scheme to exploit redundancies that may persist under different color representations. Future optimizations may also scale to more color spaces such as the BCoCg colorspace.

We additionally tested a third optimization that didn't generate any compression improvements. This third optimization picked a prediction that was the max of the pixel values in the block. In contrast, the baseline implementation selects the minimum pixel value in the block. The residuals are then calculated as prediction minus the true value. Thus, to calculate the true values, the residuals are subtracted from the predicated value, instead of being added to the prediction like in the baseline. This optimization did not yield any compression improvements and thus, left out of our QuickSend scheme.

3 Methodology

In order to provide as much flexibility as possible to test a variety of workloads on various algorithms, we present a comprehensive infrastructure. The flow is outlined in Fig. 1. We built a multi-threaded framework to be able to run millions of simulated compressions and output CSVs per image for all the algorithms, cache parameters, and access patterns tested.

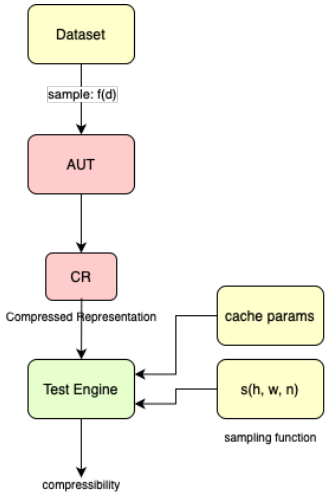

Figure 1: Compressibility testing framework. Highlighted in green is outputs, yellow are inputs, and red are internal determined information.

3.1 Dataset

The dataset is a generic tensor of dimensions (N, C, W, H), where N is the number of samples in the dataset, C is the channels per image, W is the width of an image, and H is the height of an image. We chose to order C before W and H, as opposed to traditional image formats because all the compression algorithms operate on each channel individually, and this would make it more intuitive for the application programmer.

The dataset we used for our evaluations is a set of 55 images with varying properties, including but not limited to grayscale, gradients, color patterns, and color diversity. This will yield a comprehensive test set to extract key performance properties for all the algorithms we test.

3.2 Algorithm-Under-Test (AUT) and Compressed Representation (CR)

Mimicked from SystemVerilog test benches, AUT is a flexible algorithm that accepts a set of inputs on which simulations can be run. We allow AUT to take in a dataset to operate on and provide overloadable compression functions for architects to test various compression mechanisms.

The AUT generates a compressed representation of the images in the dataset, and this is the crux of what the Test Engine operates on. The exact semantics of the CR are entirely up to the architect, as long as it is consistently sampled in the Test Engine.

3.3 Sampling Function

We define a sampling function as

$$s(h, w, n) : \mathbb{Z}^3 \rightarrow \{(x_1, y_1), ...(x_n, y_n)\}$$

Essentially, given a height and width of an image and a target number of points, the function simply returns a set of n points that are within the bounds, in accordance with a sampling policy. All the access patterns used in QuickSend evaluations are displayed in Fig. 2.

3.3.1 Random Access. The random access pattern randomly samples (x, y) pairs such that $x \in [0, W] \wedge y \in [0, H]$. This is a baseline pattern to compare against.

3.3.2 Periodic Access. Another common GPU access pattern is periodic access, e.g. when computing textures or lossy approximations. To model this, the second pattern in 2 illustrates a zig-zag like pattern.

3.3.3 Strided Access. The strided access pattern accesses pixels uniformly with a stride, e.g. when computing convolutions with strides and dilations.

3.3.4 Blurred Access. The blurred access pattern is a Gaussian distribution, and points are sampled like $(x, y) \sim N(\mu, \sigma^2)$ where μ is the center of the image and σ^2 is the variance, a representation of how local the accesses are.

3.3.5 Skew Access. Finally, the skew access pattern is a Gaussian distribution with a randomly initialized mean, and user-specified variance.

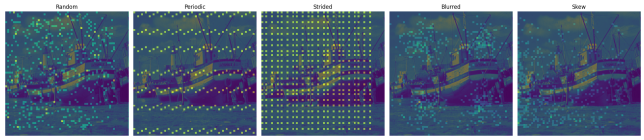

Figure 2: All the access patterns (sampling functions) used in QuickSend evaluations.

All of the access patterns mimic various use-cases and help understand the proper circumstances under which certain hardware compression algorithms may perform better or worse than others.

3.4 Caching

In our testing harness, we additionally add in the notion of GPU caches. This is important as the GPU will generally store sent blocks for some time to take advantage of spatial locality in the access pattern. To simulate a similar cache in our testing harness, we implement an LRU cache that stores recently sent blocks to the GPU. The cache is a small fully-associative cache. The testing harness can have arbitrary cache sizes that allow us to analyze compression schemes under a range of cache values.

With caches, we additionally introduce a new term, *communication ratio* which measures both the effect of the GPU cache and the compression scheme. It can be calculated as bits communicate over the bits sent, and is better when higher, meaning that more bits were communicate per bit sent. This captures both the data movement reduction from the cache and the compression schemes.

4 RTL Methodology

An important consideration to take into account when designing a hardware algorithm is the power and area metrics. To properly evaluate QuickSend, we decided to create a baseline compression algorithm against which to compare the power and area increases. We decided to use the Intel iGPU 8th Generation hardware compression algorithm as a baseline as it is well documented, straightforward to verify, and easy to tune and modify.

4.1 Baseline Design

4.1.1 iGPU 8th Generation Overview. To design the baseline algorithm, we follow the compression algorithm outlined in [14]. That is, we

(1) Compute the minimum `r,g,b,a` values in the 4×8 block
(2) Compute the residuals between all pixel channels and the minimums
(3) Determine if it is compressible if the sum of the number of bits required to store the *maximum* residuals is ≤ 14
(4) Generate a header with all the information, along with skip bits if residuals are 0
(5) Compress all the 4×8 data into a single cache line (64 bytes)

Our hardware design incorporates three pipeline stages, where inputs and outputs are combinational buffers. The input is a $4 \times 8 \times 4 = 128$ byte buffer, i.e. a single block of the image. The output is a 64 byte buffer and a flag indicating if the block was compressed or not.

Designing a single block-level compression mechanism is sufficient as that is the unit of granularity the original iGPU algorithm operates on [14], and parallelizing with multiple blocks will have predictable area and power impacts from a single block implementation. For example, processing a full 64×64 image would require 128 blocks, and the area is proportional to

$$O(\texttt{N} \cdot \texttt{area\_of\_block} + \texttt{overhead}) \quad (1)$$

, where `overhead` is logic required in the top modules for connections and $N = 128$. Power follows similar reasoning.

4.1.2 Pipelined Stages. The overall iGPU 8th Generation baseline design can be visualized in Fig. 3.

The first stage of the pipelined design is the `header` stage. This stage computes the minimum pixels and sets up the header packet with all the information but the maximum residuals and skip bits.

The second stage computes the actual residuals between each pixel and the minimum value stored within the header packet passed from the previous stage. At this point, we can determine if the data is compressible, and the `flag` is updated correspondingly.

Finally, the `compress` stage populates the output buffers with the pixels. This stage is lightweight, but we chose to extract it into a dedicated pipeline stage to reduce the critical path, which appears in the `residual` stage.

If timing constraints were stricter, we can parallelize the minimum residual computations in the `header` stage further. Currently, we parallel compute the minimum by breaking the buffer into four

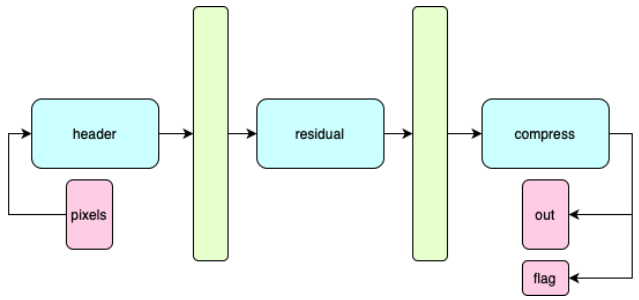

Figure 3: Three-stage pipeline for iGPU 8th Generation (reverse engineered). Teal modules are purely combinational, connected via the green pipeline registers. Inputs and outputs are highlighted in pink.

buffers, and then doing a reduction tree on the minimum in the four buffers. This can be further parallelized at a cost of higher area.

The `header` stage passes the partially constructed header packet and all the pixel data into the pipeline register for the `residual` stage. The `residual` stage passes the fully constructed header and residual pixels into the pipeline register for the `comrpess` stage.

4.2 QuickSend Design

We now present the QuickSend compression algorithm architecture, and the modifications on top of iGPU 8th Generation.

To minimize modification and retain a true baseline, we chose to incorporate the basic iGPU 8th Generation module into our design. Fig. 4 illustrates the visual components of our algorithm.

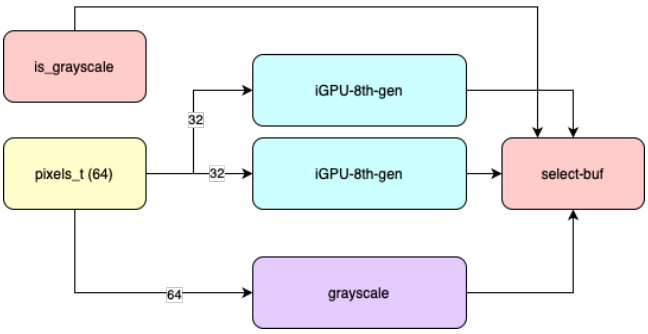

Figure 4: QuickSend compression architecture. Teal and purple modules are purely combinational, and each module requires similar area requirements as they are all modified instances of the base iGPU 8th Generation module. The input is highlighted in yellow. Combinational selection logic is highlighted in red.

Recall that QuickSend can completely compress a grayscale image into half a cache line. This is because grayscale information contains information in two channels while RGBA contains information in four channels. Therefore, to fully utilize a cache line and have full bus utilization, QuickSend must have at least 64 pixels, or double that of RGBA.

4.2.1 Architectural Modifcations. To allow for this modification, QuickSend takes in 64 pixels, or two iGPU blocks of pixels. Then, we process three computations in parallel: 2 iGPU 8th Gen on 32 pixels each, and one grayscale computation on 64 pixels.

Additionally, the core logic is padded with combinational selection logic. `is_grayscale` outputs a flag to determine if *all* 64 *pixels are grayscale*.

In order to determine if an image is grayscale, we ensure that for any pixel i, $r_i = b_i = g_i$. Essentially, the RGBA equivalent of a grayscale image broadcasts the color channel of the gray pixel across the red, green, and blue channels.

This combinational logic does not yield the critical path, and also applies a similar reduction tree logic as the `header` stage in Fig. 3.

The iGPU 8th Gen modules output either 2, 3, or 4 buffers of 64 bytes each. Two would be the best case where both modules were able to fully compress the respective 32 pixel block, and four is the worst case where neither could.

The `grayscale` module will either compress 64 pixels into a single cache line of 64 bytes or not.

`select_buf` takes in the flag from `is_grayscale` and performs the following logic in Algorithm 1. Note, there are four states the flag can be: gray compressed, one color buffer compressed, two color buffers compressed, and no compression. We apply a priority to the grayscale as it yields the highest compressibility ratio of $4:1$.

Algorithm 1 Selection logic for QuickSend

```
pixels ← arr[64][4]
gray ← is_grayscale(pixels)
gray_compressable ← INPUT
color_compressable ← INPUT
if gray ∧ gray_compressable then
    flag ← GRAY_COMPRESS
    return flag, gray_buf
end if
if color_compressable then
    flag ← COLOR_COMPRESS_{0,1}
    return flag, color_buf
end if
flag ← NO_COMPRESS
return flag, NONE
```

4.2.2 Design Impacts. The grayscale combinational logic before the iGPU modules requires an additional clock cycle. Therefore, we increased the hardware algorithm from a three stage pipeline in Fig. 3 to a four stage pipeline in Fig. 4. However, we maintain the same clock cycle of `250mHz`. Therefore, the latency of the system increases, but the throughput remains the same. We argue this is a minor impact as the primarily limiter is the rate of bus transfers and requests on the GPU side, and a full pipeline in either algorithm will perform the same.

With manageable architectural modifications, we increased the possibility of compression from a max of $2:1$ to $4:1$, and allow much more dynamic compression based on input workloads. We doubled the input size of the pixel buffers to two iGPU blocks of

pixels to fully utilize the grayscale compression, and output appropriate flags for decoding which compression mechanism was performed. At a single extra cycle of latency we managed to potentially gain a 2× increase in compression for certain workloads.

4.2.3 Synthesis. To synethsize the design, we used a `250mHz` clock and `45nm` process technology nodes. Both designs were synthesized with identical configurations and optimization passes to ensure a fair comparision between QuickSend and our baseline.

5 Evaluation

We evaluate QuickSend in two parts. Firstly, we do a theoretical compression and communication ratio evaluation by comparing it against numerous industry algorithms like Intel iGPU 8th Gen, Intel iGPU 11th Gen, and Mali. Then we conduct an area and power analysis between QuickSend and an industry baseline, iGPU 8th Gen. Note, we denote communication ratio as the ratio of bytes delivered vs. bytes communicated. Compression ratio is evaluating the communication ratio with a cache size of 0.

5.1 Compression and Communication Evaluation

We note that the general trend for compression across the algorithms, in increasing order of compression is: uncompressed, Intel 8th Gen, Intel 11th Gen, Own Grey, Own Yuva, Mali, as indicated by in Fig. 5. This is expected behaviour. Uncompressed should naturally be the lowest, as it is sending the raw data every request. Intel iGPU 8th Gen and Intel iGPU 11th Gen have a theoretical maximum compression ratio of 2.0. We improved on the Intel iGPU 8th Gen to get a maximum attainable compression ratio of 4.0 for both grayscale and YUVA. Mali has a theoretical compression ratio of 8.0.

5.1.1 Evaluation Overview. We used a sample size of $N = 50,000$ to ensure that we have reliable and replicable results. This means that every access pattern for every cache parameter for every algorithm was sampled 50, 000 times.

The cache parameters we used were:

$$C = \{0, 1, 4, 8, 16, 32, 64, 128, 256\}$$

The patterns that we used were:

$$P = \{\text{random}, \text{strided}, \text{periodic}, \text{skew}, \text{blurred}\}$$

Therefore, the total amount of compressions *per algorithm* were $N \times |C| \times |P| = 2,250,000$. For each individual test for a cache parameter and patter, we communicated around 128, 000 bytes of information.

5.1.2 Overall Compression. Note, the compression numbers in some of the boxes in Fig. 5 are extraordinarily high due to deep LRU caches.

We also note that strided yields high communication ratios with a shallow cache, indicating it can effectively take advantage of an LRU policy. The periodic access pattern also takes advantage of the cache even at low depths, but has a lower ceiling than the strided. The blurred pattern requires a deep cache, but given a sufficiently deep cache has incredibly high communication ratios. Without any caching scheme, all patterns have reasonably similar compression ratios.

5.1.3 Communication Ratio by Pattern. As indicated in Fig. 6, we note that the pattern has a significant impact on achievable compression. This aligns with the observations in Fig. 5, where strided has incredibly high performance across most cache depths, with blurred and periodic following. However, the skew and random pattern do not take advantage of the caching scheme.

5.1.4 Communication Ratio by Cache Size. Fig. 7 illustrates the communication efficiency as a function of the cache size. As expected, the deeper the cache, the better all the algorithms perform. Note that the median for Intel iGPU 8th Gen, Intel iGPU 11th Gen, and uncompressed are roughly similar. However, the means for YUVA and grayscale compression on QuickSend are noticeably higher, indicating that the mechanism was able to take advantage of grayscale patterns within the image. Finally, Mali has the highest median we observed.

5.1.5 Theoretical Compression Conclusions. We conducted an extensive design space exploration across varying access patterns and cache parameters for the algorithms. We observed that, on average, our own algorithms perform noticeable better than the baseline iGPU 8th Gen. algorithm, but around half as effective as Mali. Mali can compress more effectively, but at the cost of a more complicated tree-based algorithm. We managed to keep QuickSend a relatively simple algorithm to design, with noticeable improvements to the baseline Intel implementations.

5.2 RTL Evaluation

We will evaluate the iGPU 8th Generation RTL baseline implementation, along with the QuickSend grayscale implementation, in terms of area and power.

5.2.1 iGPU Baseline Area. The overall synthesis area report is summarized in 8. This is synthesized using Synopsys Design Compiler on `45nm` process technology nodes. We synthesized with several iterations, though noticed that after the first iteration area was not being lowered significantly. Notice that the bulk of the compression overhead comes from the `header` stage and the `compression` stage, namely the header and residual computations.

This is due to the reduction tree approach to reduce the critical path, which presents a tradeoff between critical path and area. The overall combinational area is `26283.1943` μm^2 and the sequential area is `16291.4361` μm^2. The large sequential area is a result of passing a full block of pixels to the `residual` stage, and passing a block of residuals from the `residual` stage to the `compress` stage.

The large pipeline registers were another reason why we chose to keep the pipeline shallow and just 3 stages, as a deeper pipeline would increase the area overhead. This is backed by Fig. 9, where we note that the actual sequential cell counts is low, indicating that the registers are just large and occupy lots of area.

Additionally, note that there is relatively low top module logic to properly connect all sub-modules, and the bulk of the area is the compression logic itself.

5.2.2 iGPU Baseline Power. Another important consideration in hardware design is the power consumption of the system. Our

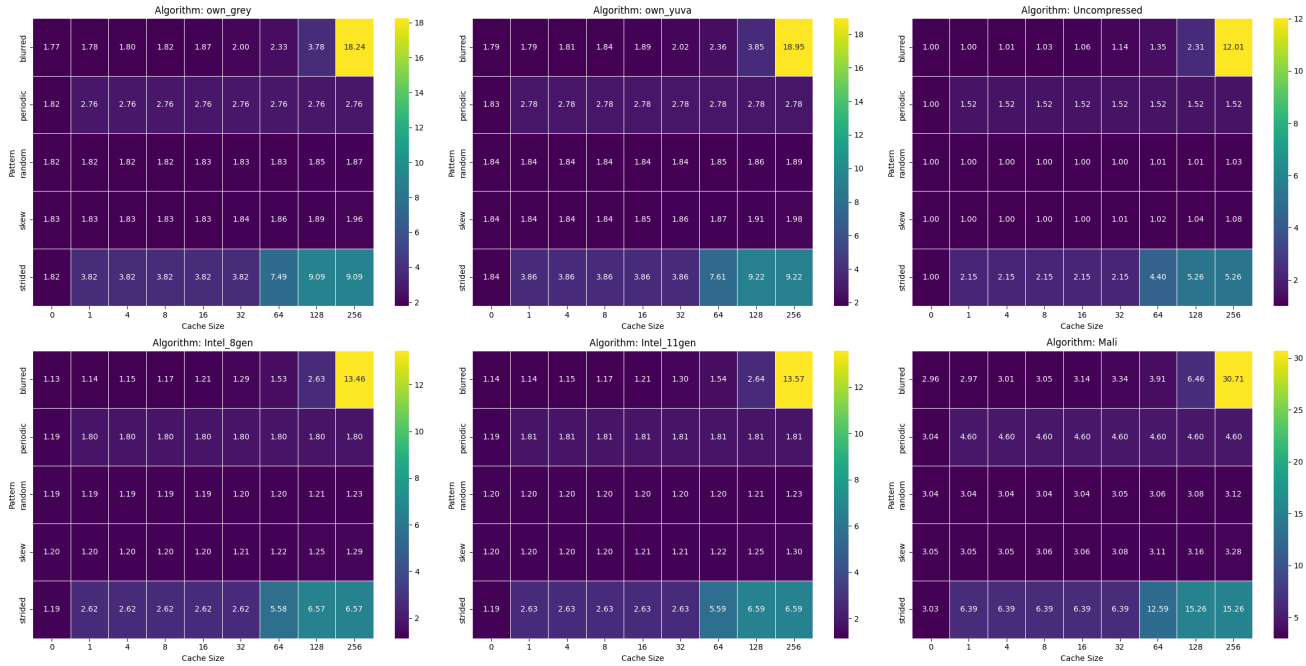

Figure 5: Heatmap of cache size vs. pattern and the observed compression.

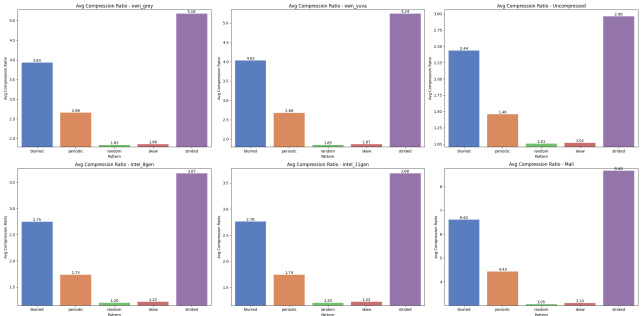

Figure 6: Communication ratio for each algorithm by pattern.

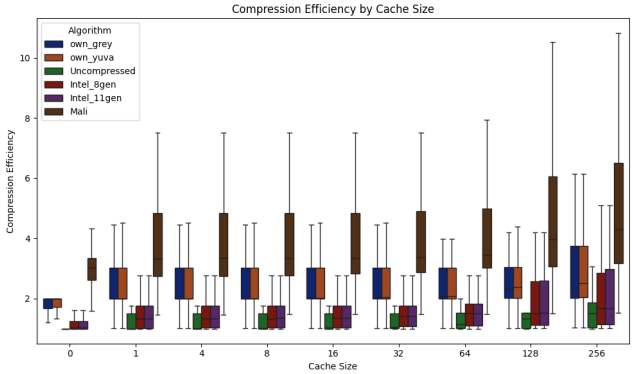

Figure 7: Communication ratio efficiency for each algorithm by cache size.

primarily goal regarding power is to have a low-overhead algorithm that compresses the data to avoid excess power in data transfers across the CPU to GPU bus.

Recall that power in hardware can be broken down into three main categories: interal, switching, and leakage. Of the three, internal and switching fall under *dynamic power*, while leakage falls under *static power*. Internal power is the power consumed by the transistors to maintain the state they are in. Switching power is the power leakage due to a split-second short when the transistors switch from low to high and vice versa.

As expected, the baseline implementation power distribution primarily consists of internal power, and leakage power is low as shown in Fig. 10. Additionally, the total power leakage is $4.3777 \times 10^3 \mu w$. This means that the baseline implementation is a low cost algorithm that resides on top of the last level cache (LLC), and can drastically reduce the power expended by the bus at little overhead costs.

5.2.3 QuickSend Area & Power. Recall the architecture in Fig. 4. The grayscale module has functionally the same input size as the iGPU baseline modules, and therefore has similar area metrics. The increase in area therefore comes from tripling the compression mechanisms and adding combinational logic to check grayscale and select.

We have a single combinational module that is responsible for this logic, and is $1380.5400\ \mu m^2$ in area. This is incredibly low-overhead in context with the rest of the QuickSend design, and illustrates that the primary design cost is simply the 64 pixels being

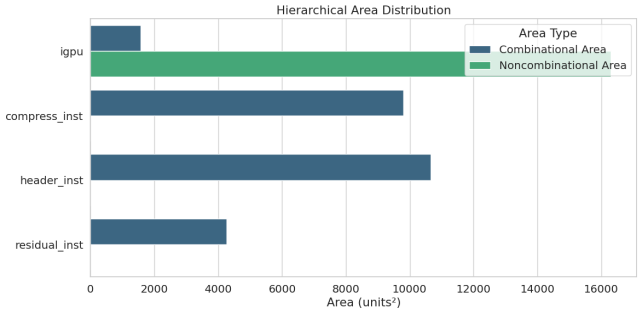

Figure 8: Synthesis Area Reports for iGPU 8th Gen Baseline

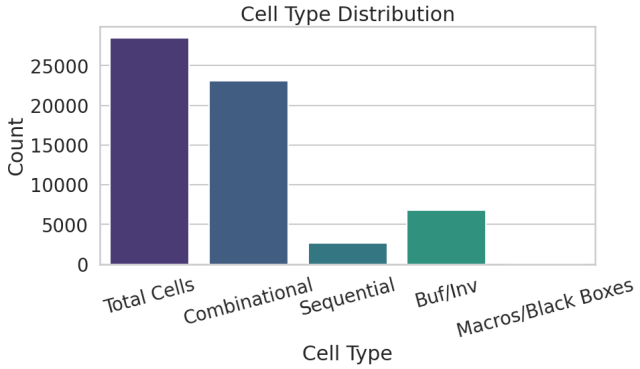

Figure 9: Synthesis Cell Reports for iGPU 8th Gen Baseline

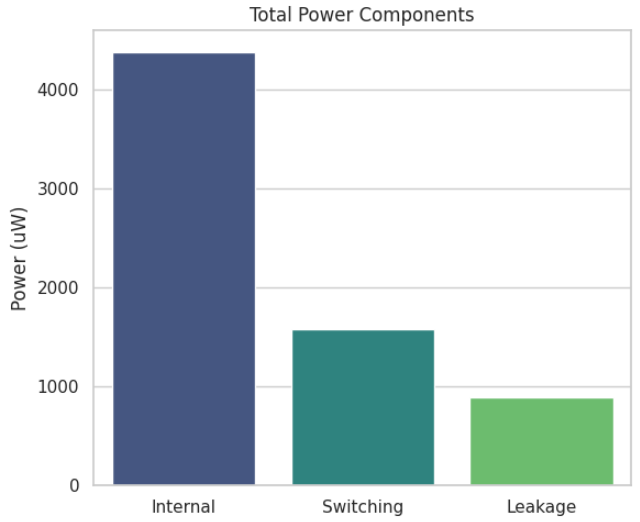

Figure 10: Synthesis Power Reports for iGPU 8th Gen Baseline

processed in parallel and the sequential logic required to pipeline the algorithm.

The total area for QuickSend is then `150877` μm^2, or a `3.67` $\times$ increase over the baseline. Though this appears as a large overhead, the baseline itself is incredibly low-overhead as it is a small part of the LLC and the overall die area. Therefore, though a large increase relatively, it still comprises of a small part of the die area and is feasible.

Moreover, at the expense of this area increase, we have introduced much more flexibility into the system and allowed $4:1$, $3:1$, and $2:1$ compression opportunities, as opposed to just $2:1$ in the baseline. The power increases follow a similar trend and still demonstrate a low cost design. Compression mechanisms like Mali allow for a much better compression, but require tree based methods and pointers, making it complicated in hardware. QuickSend maintains the iGPU 8th Gen. philosophy of using simple structures and operations, but increments on the iGPU 8th Gen. design by introducing different color spaces. We argue this wider opportunity for compression along with the manageable area and power increases make QuickSend a competitive hardware compression mechanism.

6 Related Works

Work by Wang et al. in GPU.zip [14] sheds light on two Intel GPU compression algorithms and uses them to create a hardware side channel to conduct pixel stealing attacks. This work provides a glimpse at hardware compression algorithms but does not compare the hardware compression algorithms against each other or offer novel compression schemes.

The Mali compression scheme is partially explained in the work of Nystad et al. [7]. However, this work only explains how the Mali compression scheme works and does not address a comparison across industry compression schemes. Moreover, our work provides implementations of the Mali scheme which are not provided in Nystad et al.'s work.

The Texture Compression blog post [9] conducts a thorough design space exploration of modern hardware compression algorithms used in industry. It explores standard block compression formats implemented by various companies like Intel, Apple, etc. However, all the algorithms studied are lossy algorithms and software exposed. Our work explores software-transparent lossless algorithms on a theoretical and practical level, and introduces novel modifications to improve performance.

We use LZ4 as a baseline compression mechanism [1]. The paper provides and tests an implementation of the LZ4 compression algorithm in hardware. However, we are exploring algorithms specific for textured data and between the CPU and GPU bus. The paper provides a useful baseline against which to compare, but is adjacent to the kinds of algorithms we are exploring and enhancing.

Lipsinki et al. looks at the impact of GPU-based image compression on the overall distributed rendering pipeline. Their work discusses compressibility ratios and data transfer from the GPU to main memory [6]. While this is adjacently related to our project, our focus is on effective bandwidth for CPU-GPU compression algorithms. This differs any and all of the compression algorithms we are benchmarking and using to direct our own novel algorithm.

Work by Sathish et al. [10] discusses lossless and lossy memory I/O link compression for improving performance of GPGPU workloads. We can leverage these findings when we look into the

metadata overhead and their tradeoffs. Their focus is on providing microarchitecture enhancements to support lossless and lossy compressions through GPU memory I/O links while our work introduces novel modification to improve performance within software-transparent CPU-GPU compression algorithms.

Knorr et al. introduce ndzip-gpu, an efficient GPU-parallelized lossless compression method for scientific floating-point data, achieving high throughput on NVIDIA hardware with strong compression ratios [5]. Their study indicates that ndzip-gpu excels at compressing multi-dimensional grids with strong correlations across dimensions, while unstructured datasets may benefit more from stream compressors like MPC or dictionary coders like LZ4. However, their focus is on floating-point data, whereas our work targets graphics and image data.

A parallel implementation of bzip2-like lossless compression for GPU architectures is explored by Patel et al., focusing on the parallelization of three key stages: Burrows-Wheeler transform (BWT), move-to-front transform (MTF), and Huffman coding [8]. The study suggests that current GPUs do not provide sufficient performance benefits for real-time lossless compression compared to the serial bzip2 algorithm.

7 Conclusion

As we enter an era where increasing transistor count is no longer sufficient to meet escalating compute demands, architects must adopt and explore creative approaches. In our work, we have demonstrated that hardware compression offers a promising path forward, meeting computing demands without breaking the area and power bank. Moreover, in this new era of architecture, there is a growing need to thoroughly evaluate different hardware compression schemes and derive actionable insights. Our analysis of two Intel compression schemes and one Arm Mali scheme serves as a foundation for future comparisons and analyses. Our QuickSend design highlights the potential for further improvements in compression performance. Overall, our work establishes a starting point for future hardware compression research. While hardware compression is just one of many championed hardware optimizations, hardware compression is clearly here to stay and will greatly influence the future computing architectures that will address the void left by Dennard's scaling.

8 Acknowledgments

We would like to thank Hovav Shacham for allowing us to use his reverse engineering work on the Mali GPU compression algorithm for this project.

References

[1] Matěj Bartík, Sven Ubik, and Pavel Kubalik. 2015. LZ4 compression algorithm on FPGA. In *2015 IEEE International Conference on Electronics, Circuits, and Systems (ICECS)*. 179–182. doi:10.1109/ICECS.2015.7440278

[2] Boru Chen, Yingchen Wang, Pradyumna Shome, Christopher W. Fletcher, David Kohlbrenner, Riccardo Paccagnella, and Daniel Genkin. 2024. GoFetch: Breaking Constant-Time Cryptographic Implementations Using Data Memory-Dependent Prefetchers. In *USENIX Security*.

[3] Jason Kim, Jalen Chuang, Daniel Genkin, and Yuval Yarom. 2025. FLOP: Breaking the Apple M3 CPU via False Load Output Predictions. In *USENIX Security*.

[4] Jason Kim, Daniel Genkin, and Yuval Yarom. 2025. SLAP: Data Speculation Attacks via Load Address Prediction on Apple Silicon. In *S&P*.

[5] Fabian Knorr, Peter Thoman, and Thomas Fahringer. 2021. Ndzip-gpu: Efficient Lossless Compression of Scientific Floating-Point Data on GPUs. In *International Conference for High Performance Computing, Networking, Storage and Analysis, SC*. IEEE Computer Society. doi:10.1145/3458817.3476224

[6] Riley Lipinksi, Kenneth Moreland, Michael E. Papka, and Thomas Marrinan. 2021. GPU-based Image Compression for Efficient Compositing in Distributed Rendering Applications. In *2021 IEEE 11th Symposium on Large Data Analysis and Visualization (LDAV)*. 43–52. doi:10.1109/LDAV53230.2021.00012

[7] Jorn Nystad, Oskar Flordal, Jeremy Davies, and Ola Hugosson. 2015. Methods of and Apparatus for Encoding and Decoding Data in Data Processing Systems.

[8] Ritesh A Patel, Yao Zhang, Jason Mak, Andrew Davidson, and John D Owens. [n. d.]. Parallel Lossless Data Compression on the GPU.

[9] Aras Pranckevičius. 2020. Texture compression in 2020 · aras' website. https://aras-p.info/blog/2020/12/08/Texture-Compression-in-2020/

[10] Vijay Sathish, Michael J. Schulte, and Nam Sung Kim. 2012. Lossless and lossy memory I/O link compression for improving performance of GPGPU workloads. In *Proceedings of the 21st International Conference on Parallel Architectures and Compilation Techniques* (Minneapolis, Minnesota, USA) *(PACT '12)*. Association for Computing Machinery, New York, NY, USA, 325–334. doi:10.1145/2370816.2370864

[11] Jose Rodrigo Sanchez Vicarte, Michael Flanders, Riccardo Paccagnella, Grant Garrett-Grossman, Adam Morrison, Christopher W. Fletcher, and David Kohlbrenner. 2022. Augury: Using Data Memory-Dependent Prefetchers to Leak Data at Rest. In *IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society.

[12] Jose Rodrigo Sanchez Vicarte, Pradyumna Shome, Nandeeka Nayak, Caroline Trippel, Adam Morrison, David Kohlbrenner, and Christopher W. Fletcher. 2021. Opening pandora's box: a systematic study of new ways microarchitecture can leak private data. In *Proceedings of the 48th Annual International Symposium on Computer Architecture* (Virtual Event, Spain) *(ISCA '21)*. IEEE Press, 347–360. doi:10.1109/ISCA52012.2021.00035

[13] Alan Wang, Boru Chen, Yingchen Wang, Christopher Fletcher, Daniel Genkin, David Kohlbrenner, and Riccardo Paccagnella. 2025. Peek-a-Walk: Leaking Secrets via Page Walk Side Channels . In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, Los Alamitos, CA, USA, 23–23. doi:10.1109/SP61157.2025.00023

[14] Yingchen Wang, Riccardo Paccagnella, Zhao Gang, Willy R. Vasquez, David Kohlbrenner, Hovav Shacham, and Christopher W. Fletcher. 2024. GPU.zip: On the Side-Channel Implications of Hardware-Based Graphical Data Compression. In *2024 IEEE Symposium on Security and Privacy (SP)*. 3716–3734. doi:10.1109/SP54263.2024.00084